Bézier Curve: Quadratic (2-D)

Plots.jl Visualisation

Code and Plot Explanation for Julia Beginners

`11-bezier-quadratic-plots.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `11-bezier-quadratic-plots.jl` — the first plotting script in the series. It covers both what the plot shows and how the code produces it, from scratch for Julia beginners. The script uses **Plots.jl**, one of Julia's two major plotting libraries.

Contents

1 The Plot **3**

2 Reading the Plot **3**
2.1 The three visual elements . . . 3
2.2 The annotations . . . 4
2.3 Geometric observations . . . 4

3 What is Plots.jl? **4**

4 Type Map **5**

5 Data and Constants **5**

6 The `BezierQuadratic` Struct and Constructor **6**

7 The Math Functions **6**
7.1 `basis(u)` — Bernstein weights . . . 6
7.2 `point(bz, u)` — one point on the curve . . . 7

7.3 `curve(bz, n)` — the full curve as a matrix 7
7.4 `highlight_points(bz, u_list)` 8

8 Annotation Offsets **8**

9 `draw_and_save` — Building the Plot **8**
9.1 Creating the base figure 9
9.2 Adding the curve 9
9.3 Adding the control polygon 10
9.4 Annotating control points 11
9.5 Adding the highlighted points 11
9.6 Annotating highlighted points 12
9.7 Adding the equation text 13
9.8 Saving and displaying 13

10 Entry Point **13**

1. The Plot

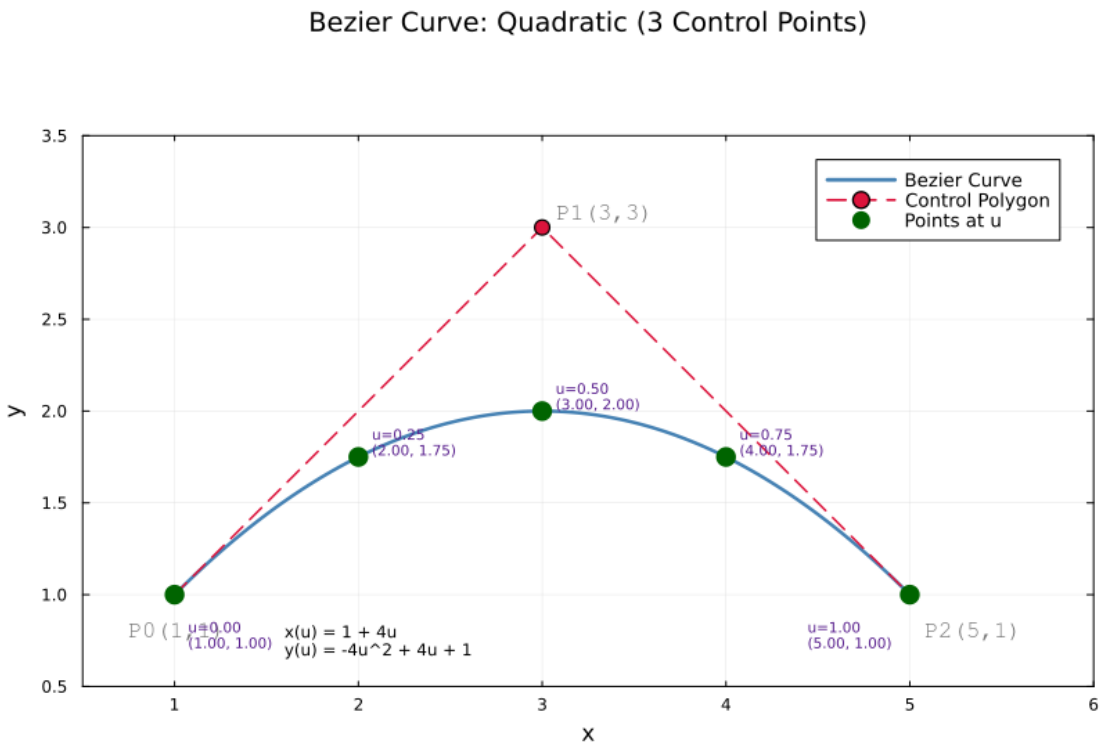


Figure 1: Quadratic Bézier curve with 3 control points, plotted with Plots.jl.

2. Reading the Plot

Before looking at a single line of code, let us understand what is being shown.

2.1. The three visual elements

The plot contains three distinct visual layers, each with its own entry in the legend:

Blue solid line — Bezier Curve. This is the actual curve: 200 points computed by evaluating the parametric equations at $u = 0, \frac{1}{199}, \frac{2}{199}, \ldots, 1$ and connecting them. The curve is smooth, starts at $P_0 = (1, 1)$, and ends at $P_2 = (5, 1)$. It does not pass through $P_1 = (3, 3)$ — that point only pulls the curve upward.

Red dashed line with circles — Control Polygon. The straight lines connecting the three control points $P_0 \to P_1 \to P_2$ form the control polygon. The red circles mark the exact positions of P_0, P_1, P_2. The control polygon is the skeleton that shapes the curve.

Green dots — Points at u. Five specific points on the curve are highlighted at $u = 0, 0.25, 0.50, 0.75, 1.00$. Each green dot sits exactly on the blue curve.

2.2. The annotations

Control point labels (red, bold). `P0(1,1)`, `P1(3,3)`, `P2(5,1)` label each control point.

Highlighted point labels (purple). Each green dot is annotated with its u value and coordinates, for example `u=0.50 / (3.00, 2.00)`. The offsets are carefully chosen to avoid overlapping the curve or each other.

Parametric equations (black, bottom-left). The two polynomials are written directly on the plot:

$$x(u) = 1 + 4u \qquad y(u) = -4u^2 + 4u + 1$$

These are the same polynomials derived algebraically in script 01.

2.3. Geometric observations

The curve is symmetric. The control points $P_0 = (1, 1)$ and $P_2 = (5, 1)$ are at the same height, and $P_1 = (3, 3)$ is exactly centred between them at $x = 3$. This produces a perfectly symmetric arc.

The midpoint $u = 0.5$ is the peak. At $u = 0.5$: $x = 1 + 4(0.5) = 3$, $y = -4(0.25) + 4(0.5) + 1 = 2$. The green dot at $(3, 2)$ is the highest point on the curve.

Points $u = 0.25$ and $u = 0.75$ are mirror images. Both sit at $y = 1.75$, symmetric about $x = 3$. At $u = 0.25$: $x = 2$, $y = 1.75$. At $u = 0.75$: $x = 4$, $y = 1.75$.

The curve always lies within the control polygon. This is the convex hull property of Bézier curves. Every point on the curve is a weighted average of the control points, so it stays inside the triangle formed by P_0, P_1, P_2.

3. What is Plots.jl?

Julia has two major plotting libraries. Understanding which is which helps you read documentation and choose the right tool.

Plots.jl is a *plotting metapackage*. It provides a single, unified API and uses a separate *backend* to do the actual rendering. You write `plot(x, y)` and Plots.jl translates that into calls to GR, PyPlot, Plotly, or other backends. This means:

- One syntax works across multiple backends.
- The default backend (GR) requires no extra setup.
- It is well-suited for static plots saved to PNG or PDF.

Makie.jl (used in scripts 11-makie, 12, 21, 22) is a *native Julia plotting framework*. It renders directly without a backend, supports interactive and GPU-accelerated plots, and has a different API. CairoMakie renders to files; GLMakie renders in a window.

Which should you use?

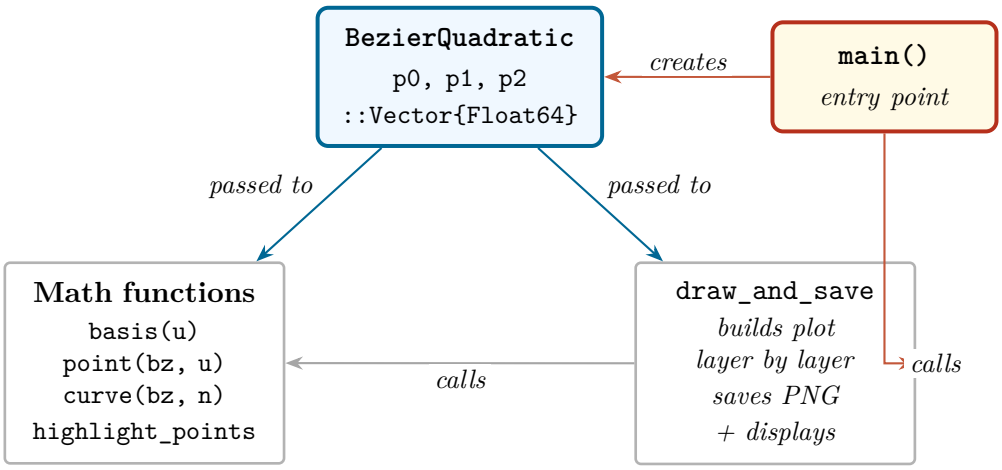
Use **Plots.jl** when you want a simple, familiar matplotlib-like interface and static output.

Use **Makie.jl** when you need interactivity, animations, or high-performance rendering.

This script uses Plots.jl. Install it once with:

```
import Pkg; Pkg.add("Plots")
```

4. Type Map



This script has no `BezierExpansion` or polynomial type. The math is simpler: one struct holds the three control point vectors, three functions compute curve geometry, and `draw_and_save` handles all plotting.

5. Data and Constants

```
const CONTROL_POINTS = [
    1.0  1.0;   # P0
    3.0  3.0;   # P1
    5.0  1.0;   # P2
]

const U_HIGHLIGHT = [0.00, 0.25, 0.50, 0.75, 1.00]
```

`CONTROL_POINTS` is a 3×2 matrix. Each row is one control point; column 1 is x, column 2 is y. The `const` keyword declares a module-level constant — its value is fixed for the lifetime of the program.

`U_HIGHLIGHT` is the list of u values at which the green dots will be placed.

```
const COLOR_CURVE   = :steelblue
```

```
const COLOR_POLYGON = :crimson
const COLOR_POINT   = :darkgreen
const COLOR_LABEL   = :purple4
const COLOR_CTRL    = :crimson
```

Colours in Plots.jl are named with **Symbol** values (starting with `:`). A symbol like `:steelblue` is an interned string — Plots.jl recognises CSS colour names and a wide set of named colours. Collecting all colour choices in named constants at the top of the file makes them easy to change without hunting through the drawing code.

6. The `BezierQuadratic` Struct and Constructor

```
struct BezierQuadratic
    p0::Vector{Float64}
    p1::Vector{Float64}
    p2::Vector{Float64}
end

BezierQuadratic(pts::Matrix{Float64}) =
    BezierQuadratic(pts[1,:], pts[2,:], pts[3,:])
```

The struct stores the three control points as separate vectors rather than a matrix. This makes the math functions easier to read: `bz.p0`, `bz.p1`, `bz.p2` are immediately meaningful.

The outer constructor extracts rows from the matrix using **row-slice** syntax: `pts[1,:]` means "row 1, all columns" — a `Vector{Float64}` of length 2 (the x and y coordinates of P_0).

```
pts = CONTROL_POINTS

pts[1,:]  = [1.0, 1.0]   (P0)

pts[2,:]  = [3.0, 3.0]   (P1)

pts[3,:]  = [5.0, 1.0]   (P2)
```

7. The Math Functions

7.1. `basis(u)` — Bernstein weights

```
function basis(u::Float64)
    b0 = (1 - u)^2
    b1 = 2 * u * (1 - u)
    b2 = u^2
    return b0, b1, b2
end
```

Returns a **tuple** of three values — the three quadratic Bernstein basis polynomials

evaluated at u:

$$B_0(u) = (1-u)^2, \quad B_1(u) = 2u(1-u), \quad B_2(u) = u^2$$

In Julia, `return b0, b1, b2` returns a `Tuple{Float64, Float64, Float64}`. The caller unpacks it with `b0, b1, b2 = basis(u)`.

> `basis(0.5)` → `(0.25, 0.5, 0.25)`
>
> Check: $0.25 + 0.5 + 0.25 = 1.0$ ✓(partition of unity)

7.2. `point(bz, u)` — one point on the curve

```
function point(bz::BezierQuadratic, u::Float64)
    b0, b1, b2 = basis(u)
    b0 .* bz.p0 .+ b1 .* bz.p1 .+ b2 .* bz.p2
end
```

Evaluates the curve at parameter u:

$$P(u) = B_0(u) \cdot P_0 + B_1(u) \cdot P_1 + B_2(u) \cdot P_2$$

The dot operators `.*` and `.+` are **broadcast** operations — they apply element-wise to vectors. `b0 .* bz.p0` multiplies the scalar `b0` by each element of the vector `bz.p0`, giving a scaled 2-D vector. The three scaled vectors are then added element-wise.

The result is a `Vector{Float64}` of length 2: the $[x, y]$ coordinates of the curve at u.

> At $u = 0.5$:
>
> `b0, b1, b2 = 0.25, 0.5, 0.25`
>
> `0.25 .* [1,1] .+ 0.5 .* [3,3] .+ 0.25 .* [5,1]`
>
> $= [0.25, 0.25] + [1.5, 1.5] + [1.25, 0.25]$
>
> $= [3.0, 2.0]$ ✓

7.3. `curve(bz, n)` — the full curve as a matrix

```
function curve(bz::BezierQuadratic, n::Int=200)
    u_vals = LinRange(0.0, 1.0, n)
    pts    = hcat([point(bz, u) for u in u_vals]...)'
    return pts
end
```

`LinRange(0.0, 1.0, n)`: Creates n evenly-spaced values from 0 to 1 inclusive. With $n = 200$: $[0, \frac{1}{199}, \frac{2}{199}, \ldots, 1]$. `LinRange` is memory-efficient — it does not allocate a full array but computes each value on demand.

The list comprehension: `[point(bz, u) for u in u_vals]` produces a vector of 200 two-element vectors — one per u value.

`hcat(.....)'`: This is the most compact part. `hcat(v1, v2, ...)` **horizontally concatenates** vectors into a matrix. The `...` (splat operator) unpacks the list of

200 vectors into 200 separate arguments to `hcat`. The result is a 2×200 matrix. Transposing with `'` gives a 200×2 matrix — 200 rows, each a point (x, y).

> With $n = 3$ for simplicity:
>
> `u_vals = [0.0, 0.5, 1.0]`
>
> Comprehension gives: `[[1.0,1.0], [3.0,2.0], [5.0,1.0]]`
>
> `hcat([1.0,1.0], [3.0,2.0], [5.0,1.0])` $= \begin{bmatrix} 1 & 3 & 5 \\ 1 & 2 & 1 \end{bmatrix}$ (shape 2×3)
>
> After `'` (transpose): shape 3×2 — three rows, each a (x, y) point.

7.4. `highlight_points(bz, u_list)`

```
function highlight_points(bz::BezierQuadratic,
                          u_list::Vector{Float64})
    hcat([point(bz, u) for u in u_list]...)'
end
```

Identical in structure to `curve`, but takes an explicit list of u values instead of generating them. Returns an $m \times 2$ matrix of the m highlighted points. Used to place the green dots at $u = 0, 0.25, 0.5, 0.75, 1$.

8. Annotation Offsets

```
const U_OFFSETS = Dict(
    0.00 => ( 0.08, -0.22),
    0.25 => ( 0.08,  0.08),
    0.50 => ( 0.08,  0.08),
    0.75 => ( 0.08,  0.08),
    1.00 => (-0.55, -0.22),
)
```

Each annotation label is placed at an offset (dx, dy) from the point it annotates. The offsets are hand-tuned to prevent labels from overlapping the curve or each other.

The $u = 0$ label shifts down (`dy = -0.22`) because the curve starts at the bottom-left and there is room below. The $u = 1$ label shifts left (`dx = -0.55`) to avoid going off the right edge of the plot. All intermediate labels shift up-right (`0.08, 0.08`).

9. `draw_and_save` — Building the Plot

This is the main plotting function. It builds the figure layer by layer, with each `plot!` *or* `scatter!` *call adding one visual element. The exclamation mark* `!` *is a Julia convention meaning "modify in place" — it adds to the existing plot rather than creating a new one.*

9.1. Creating the base figure

```
plt = plot(
    size         = (800, 600),
    background   = :white,
    grid         = true,
    gridalpha    = 0.3,
    gridcolor    = :lightgray,
    framestyle   = :box,
    xlims        = (0.5, 6.0),
    ylims        = (0.5, 3.5),
    aspect_ratio = :equal,
    xlabel       = "x",
    ylabel       = "y",
    title        = "Bezier Curve: Quadratic (3 Control Points)",
    titlefontsize = 13,
    labelfontsize = 11,
    legend       = :topright,
    legendfontsize = 9,
)
```

`plot()` with no data creates an empty figure with all the layout settings applied. Key attributes:

- `size = (800, 600)` — pixel dimensions of the saved PNG.
- `framestyle = :box` — draws a box around all four sides, giving the plot the framed appearance visible in the output.
- `aspect_ratio = :equal` — forces the x and y axes to use the same scale per unit, so the curve is not distorted. Without this, a circle would look like an ellipse.
- `xlims`, `ylims` — set the visible range. The data goes from $x = 1$ to $x = 5$ and $y = 1$ to $y = 3$; these limits add padding around it.
- `legend = :topright` — places the legend in the top-right corner, where there is space between the curve and the control polygon.

9.2. Adding the curve

```
cpts = curve(bz)
plot!(plt,
    cpts[:,1], cpts[:,2],
    color     = COLOR_CURVE,
    linewidth = 2.5,
    label     = "Bezier Curve",
)
```

`cpts[:,1]` is the column-slice for all x coordinates (column 1 of the 200×2 matrix). `cpts[:,2]` is all y coordinates. Passing two vectors to `plot!` draws a line through those points.

`label = "Bezier Curve"` is what appears in the legend. If you pass `label = ""` or `label = false`, the element is excluded from the legend.

Actual content of `cpts` (first 5 and last 2 rows shown):

`cpts` is a 200×2 matrix. Each row is one (x, y) point on the curve.

Row	`cpts[:,1]` (x)	`cpts[:,2]` (y)
1	1.0000	1.0000
2	1.0402	1.0793
3	1.0804	1.1571
4	1.1206	1.2334
5	1.1608	1.3082
⋮	⋮	⋮
100	3.0000	2.0000
⋮	⋮	⋮
199	4.9598	1.0793
200	5.0000	1.0000

`cpts[:,1] = [1.0, 1.040, 1.080, ..., 3.0, ..., 5.0]` — the x vector passed to `plot!`

`cpts[:,2] = [1.0, 1.079, 1.157, ..., 2.0, ..., 1.0]` — the y vector passed to `plot!`

9.3. Adding the control polygon

```
cp = vcat(bz.p0', bz.p1', bz.p2')
plot!(plt,
    cp[:,1], cp[:,2],
    color      = COLOR_POLYGON,
    linestyle  = :dash,
    linewidth  = 1.5,
    marker     = :circle,
    markersize = 6,
    label      = "Control Polygon",
)
```

`bz.p0'` transposes the length-2 vector `[1.0, 1.0]` into a 1×2 row vector. `vcat(bz.p0', bz.p1', bz.p2')` stacks three row vectors into a 3×2 matrix — exactly the shape needed to plot the polygon.

`marker = :circle` draws a circle at each data point (the control points). `linestyle = :dash` produces the dashed red line connecting them. By setting both `color` and `marker` on the same `plot!` call, the line and the markers share the same colour — which is why the control polygon line and dots are both crimson.

Actual content of `cp`:

$$\texttt{cp} = \begin{bmatrix} 1.0 & 1.0 \\ 3.0 & 3.0 \\ 5.0 & 1.0 \end{bmatrix} \quad \text{shape } 3 \times 2$$

`cp[:,1] = [1.0, 3.0, 5.0]` — x coordinates of P_0, P_1, P_2

`cp[:,2] = [1.0, 3.0, 1.0]` — y coordinates of P_0, P_1, P_2

These two vectors define the three corners of the dashed red control polygon. Plots.jl connects them in order: $(1, 1) \to (3, 3) \to (5, 1)$.

9.4. Annotating control points

```
offsets_ctrl = [(-0.25, -0.20), (0.08, 0.08), (0.08, -0.20)]
names_ctrl   = ["P0(1,1)", "P1(3,3)", "P2(5,1)"]
for (pt, name, (dx, dy)) in zip(eachrow(cp), names_ctrl, offsets_ctrl)
    annotate!(plt,
        pt[1] + dx, pt[2] + dy,
        text(name, COLOR_CTRL, :left, 9, :bold)
    )
end
```

`eachrow(cp)`: Returns an iterator over the rows of the matrix `cp`. Each iteration gives one row as a vector — one control point.

`zip(eachrow(cp), names_ctrl, offsets_ctrl)`: Zips three iterables together. Each iteration yields a tuple of one row, one name string, and one offset pair. The pattern `(dx, dy)` in the loop variable **destructures** the offset tuple directly.

`annotate!(plt, x, y, text(...))`: Places a text label at position `(x, y)`. The `text()` function specifies the string, colour, alignment (`:left`), font size (9), and style (`:bold`).

9.5. Adding the highlighted points

```
hpts = highlight_points(bz, u_highlight)
scatter!(plt,
    hpts[:,1], hpts[:,2],
    color             = COLOR_POINT,
    markersize        = 8,
    markerstrokewidth = 0,
    label             = "Points at u",
)
```

`scatter!` adds a scatter plot (markers only, no line). `markerstrokewidth = 0` removes the border around each marker, giving the filled green circles visible in the plot.

Actual content of `hpts`:

$$\texttt{hpts} = \begin{bmatrix} 1.00 & 1.00 \\ 2.00 & 1.75 \\ 3.00 & 2.00 \\ 4.00 & 1.75 \\ 5.00 & 1.00 \end{bmatrix} \quad \text{shape } 5 \times 2$$

Each row is one green dot. Reading down: $u = 0, 0.25, 0.50, 0.75, 1.00$.

`hpts[:,1] = [1.0, 2.0, 3.0, 4.0, 5.0]` — x coords of the five dots

`hpts[:,2] = [1.0, 1.75, 2.0, 1.75, 1.0]` — y coords of the five dots

The y values confirm the symmetry: $u = 0.25$ and $u = 0.75$ both give $y = 1.75$, and $u = 0.50$ gives the peak at $y = 2.00$.

9.6. Annotating highlighted points

```
for u in u_highlight
    pt     = point(bz, u)
    x, y   = pt[1], pt[2]
    dx, dy = get(U_OFFSETS, u, (0.08, 0.08))
    lbl    = @sprintf("u=%.2f\n(%.2f, %.2f)", u, x, y)
    annotate!(plt,
        x + dx, y + dy,
        text(lbl, COLOR_LABEL, :left, 7)
    )
end
```

For each u value, this computes the point, looks up the offset from `U_OFFSETS` (falling back to `(0.08, 0.08)` if not found), formats the label string with two lines using `n` for a line break, and places the annotation.

`@sprintf("u=%.2f n(%.2f, %.2f)", u, x, y)` formats the u value and coordinates each to 2 decimal places, producing strings like `"u=0.50 n(3.00, 2.00)"`.

Value of `pt = point(bz, u)` for each iteration:

u	`pt[1]` (x)	`pt[2]` (y)	`lbl` string	offset (dx, dy)
0.00	1.00	1.00	`"u=0.00\n(1.00, 1.00)"`	$(+0.08, -0.22)$
0.25	2.00	1.75	`"u=0.25\n(2.00, 1.75)"`	$(+0.08, +0.08)$
0.50	3.00	2.00	`"u=0.50\n(3.00, 2.00)"`	$(+0.08, +0.08)$
0.75	4.00	1.75	`"u=0.75\n(4.00, 1.75)"`	$(+0.08, +0.08)$
1.00	5.00	1.00	`"u=1.00\n(5.00, 1.00)"`	$(-0.55, -0.22)$

Each annotation is placed at `(pt[1]+dx, pt[2]+dy)`. For $u = 0.50$: annotation at $(3.00 + 0.08,\ 2.00 + 0.08) = (3.08, 2.08)$, just above and to the right of the green dot.

9.7. Adding the equation text

```
annotate!(plt,
    1.6, 0.75,
    text("x(u) = 1 + 4u\ny(u) = -4u^2 + 4u + 1",
         :black, :left, 8)
)
```

A free-floating text annotation at position $(1.6, 0.75)$, just below the main plot area. The coordinates are in data units (the same coordinate system as the axes), not pixels.

9.8. Saving and displaying

```
savefig(plt, filename)
println("Saved: $filename")
display(plt)
println("Press Enter to close...")
readline()
```

`savefig(plt, filename)` saves the plot to a PNG file. The filename extension `.png` tells Plots.jl to use the PNG format.

`display(plt)` opens the plot in a window (using the active backend's display system).

`readline()` pauses execution and waits for the user to press Enter. Without this, the script would exit immediately and the window would close before you could see the plot.

10. Entry Point

```
function main()
    bz = BezierQuadratic(CONTROL_POINTS)
    draw_and_save(bz, U_HIGHLIGHT, "11-bezier-quadratic-plots.png")
end

main()
```

`BezierQuadratic(CONTROL_POINTS)` calls the outer constructor, extracting the three row vectors from the matrix.

`draw_and_save` does everything else: computes the curve, adds all plot layers, saves the PNG, and waits for the user.

How to run:

```
julia 11-bezier-quadratic-plots.jl
```

The script displays the plot in a window and saves `11-bezier-quadratic-plots.png` in the current directory. Press Enter in the terminal to close the window and exit.

Explanation for `11-bezier-quadratic-plots.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated